CSE425 — CONCEPTS OF PROGRAMMING LANGUAGES

Formal Grammar

How We Describe, Generate, and Recognize the Syntax of Programming Languages

---

PRESENTER

Instructor

DATE

Lecture 03

MOTIVATION

Why Do We Need Formal Grammar?

Because natural language rules are too vague for machines

- **The Problem**: If I say ‘put the thing over there’, a human understands — a computer sees only characters. We need **exact rules** that leave zero ambiguity.
- **The Need**: We must tell a compiler, with mathematical precision, **which strings are valid programs** and which are not.
- **The Solution**: A **formal grammar** is a finite set of rules that can generate (and therefore describe) an infinite set of valid strings.
- **The Payoff**: Once grammar is formal, we can build **automata** that mechanically recognize valid code — this is exactly what lexers and parsers do.

> KEY TAKEAWAY
>
> *A grammar is a recipe: start with a symbol, keep applying rules, and every sentence you can cook up is a valid sentence of the language.*

MOTIVATION

A Day in the Life of a Compiler

Where grammar shows up at every stage

- **Lexing**: Regex (a regular grammar) groups characters into tokens — ‘print’, ‘x’, ‘+’, ‘42’.
- **Parsing**: A context-free grammar groups tokens into statements and expressions.
- **Semantic Analysis**: The parse tree is checked against rules that a grammar cannot express.
- **Code Generation**: The tree’s shape — determined by the grammar — dictates the target code emitted.

> KEY TAKEAWAY
>
> *A compiler is a pipeline of grammar-driven transformations. Every stage consumes the output of the previous one and re-validates structure.*

MENTAL MODEL

Grammar as a Recipe

Intuition before formalism

- **Start Symbol**: The dish you want to cook (e.g. ‘a program’).
- **Non-Terminals**: Intermediate preparations (e.g. ‘the sauce’, ‘the main’).
- **Terminals**: Final ingredients you never break down further (e.g. ‘salt’, ‘the + character’).
- **Productions**: The steps — ‘to make a program, make a statement, then more statements’.

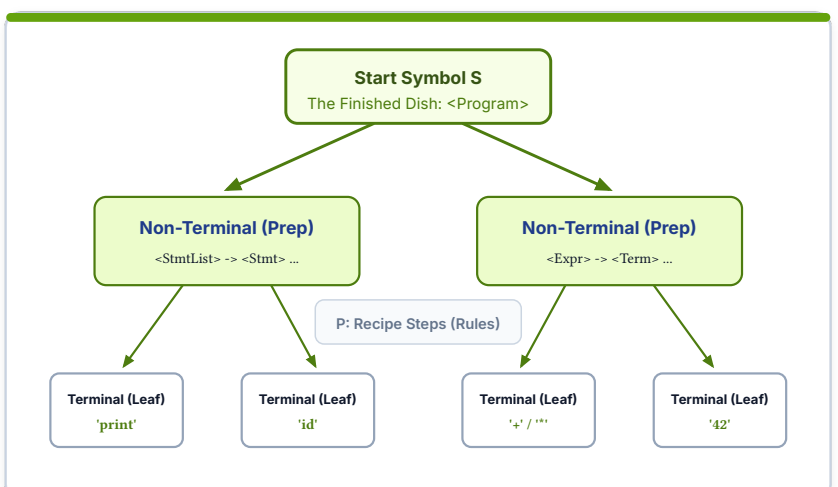


Figure: Grammar recipe — start symbol is the dish; productions are steps; terminals are the ingredients.

FOUNDATIONS

What Is a Formal Grammar? The 4-Tuple

The mathematical definition

- **Formally**: $G = (V, T, P, S)$ — four components that fully define a language.
- V — **Non-Terminals**: A finite set of placeholders or categories (`Expr`, `Stmt`, `Term`).
- T — **Terminals**: A finite set of actual symbols that appear in the final string (a, b, if, +).
- P — **Productions**: A finite set of rewrite rules of the form $\alpha \to \beta$ where α contains at least one non-terminal.

> KEY TAKEAWAY
>
> *V and T are disjoint sets. S in V is the unique start symbol. Every derivation begins at S.*

FOUNDATIONS

The Four Components Visualized

Anatomy of a grammar

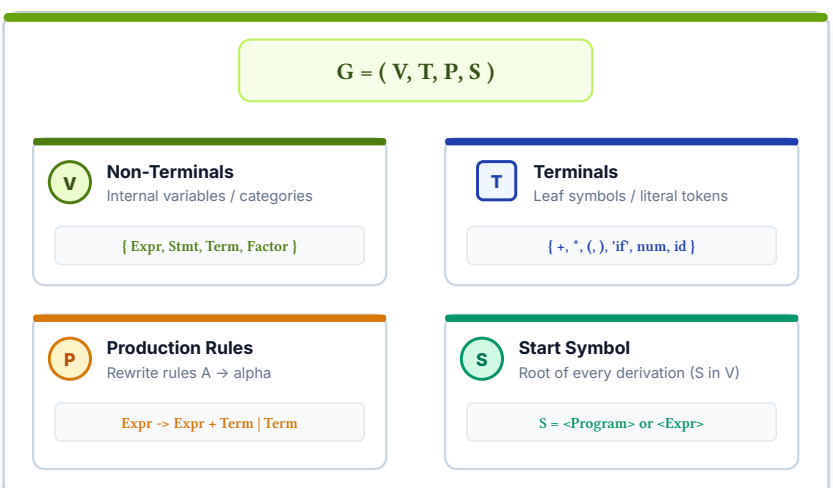


Figure: Anatomy — V (non-terminals), T (terminals), P (productions), S (start).

- **Non-Terminals (V)**: Round nodes in a parse tree — internal structure.
- **Terminals (T)**: Square leaves — the final string.
- **Productions (P)**: Arrows in the tree — each arrow is one rule application.
- **Start Symbol (S)**: The root of the tree — where every derivation begins.

TERMINOLOGY

The Vocabulary You Must Know

Six terms that unlock every grammar discussion

- **Alphabet (Σ)**: The finite set of raw symbols — for a binary language, $\Sigma = \{0, 1\}$.
- **String**: A finite sequence of symbols from the alphabet — e.g. 0110, or ‘print x’.
- **Language (L)**: A set of strings over Σ — e.g. ‘all strings with equal 0s and 1s’.
- ε: The empty string of length 0 — not the same as the empty language $\varnothing$.

> KEY TAKEAWAY
>
> *A grammar defines a language. A language is just a set of strings. Everything else is machinery on top of these three ideas.*

TERMINOLOGY

String vs Language vs Grammar

Three levels of abstraction

Concrete Level

- **String**: One specific sequence — ‘abba’, ‘print 42’.
- **Alphabet**: The raw symbols available — a, b, 0, 1, print.
- **Membership**: A string either is or is not in the language.

Abstract Level

- **Language**: The infinite set of all valid strings.
- **Grammar**: The finite rule set that defines the language.
- **Closure**: Languages are closed under union, concatenation, and star.

Synthesis: A grammar is a finite description of an infinite language. This compression is the entire reason grammars exist.

CORE MECHANISM

The Central Idea: Derivation

Generating a string is a game of substitution

- **Derivation**: Start from S. Repeatedly pick a non-terminal and replace it with the right-hand side of one of its production rules.
- **The Goal**: Keep substituting until no non-terminals remain — what is left is a string in the language.
- **Example Grammar**: $S \to aSb$ and $S \to \varepsilon$. This says: 'wrap an a and b around an S, or stop.'
- **Generated Language**: This grammar produces $\{a^n b^n \mid n \geq 0\} = \varepsilon, ab, aabb, aaabbb$, and so on.

> KEY TAKEAWAY
>
> *Derivation is the grammar producing a string. Recognition (parsing) is the reverse: given a string, can the grammar produce it?*

CORE MECHANISM

Step-by-Step Derivation Visualized

Deriving 'aabb' from $S -> a S b$ | $epsilon$

- **Step 0**: S — begin at the start symbol.
- **Step 1**: aSb — apply $S \to aSb$.
- **Step 2**: $aaSbb$ — apply $S \to aSb$ again.
- **Step 3**: $aabb$ — apply $S \to \varepsilon$ to finish.

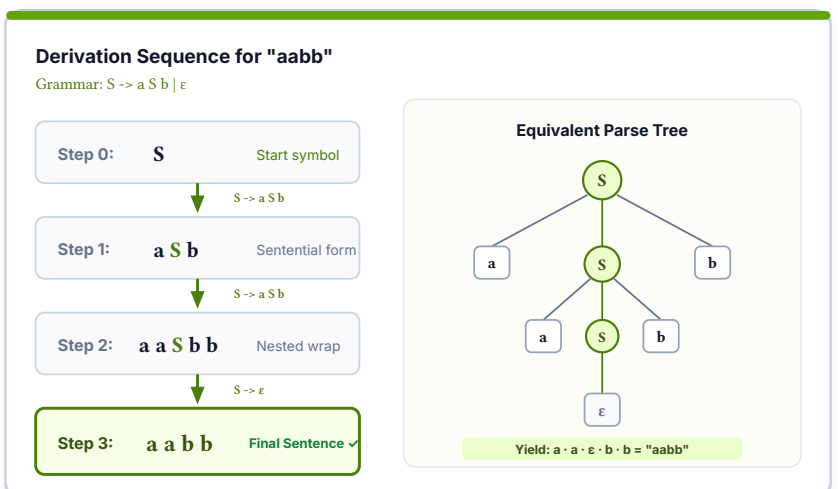


Figure: Substitution sequence — each arrow is one production application.

DERIVATION TYPES

Leftmost vs Rightmost Derivation

Order matters for parsers

- **Leftmost Derivation**: Always expand the leftmost non-terminal first — deterministic order, easy for top-down parsers.
- **Rightmost Derivation**: Always expand the rightmost non-terminal first — used in bottom-up parsing (LR parsers).
- **Same Tree, Different Order**: Both derivations produce the same parse tree — only the substitution order differs.
- **Why It Matters**: Parsers must commit to one order to be deterministic; the other is used for analysis.

> KEY TAKEAWAY
>
> *If a sentence has exactly one leftmost derivation, the grammar is unambiguous. Two leftmost derivations means ambiguity.*

INTUITION

Generation vs Recognition

Two directions of the same relationship

Generation (Grammar to String)

- **Forward direction**: Start at S, apply rules, produce a valid sentence.
- **Use case**: Defining a language — what is allowed.
- **Human analogy**: Following a recipe to cook a dish.

Recognition (String to Grammar)

- **Backward direction**: Given a string, decide if it belongs to the language.
- **Use case**: Parsing source code — is this valid?
- **Human analogy**: Tasting a dish and figuring out the recipe.

Synthesis: Every parser is a recognition machine built from a grammar. Every grammar is a generation machine that defines what the parser accepts.

TERMINOLOGY

Sentential Form vs Sentence

Half-cooked vs fully cooked

- **Sentential Form**: Any string of terminals AND non-terminals reachable from S — a partially cooked dish.
- **Sentence**: A sentential form with only terminals — a fully cooked dish, i.e. a member of the language.
- **Leftmost Derivation**: Always expand the leftmost non-terminal first — deterministic order.
- **Rightmost Derivation**: Always expand the rightmost non-terminal first — used in bottom-up parsing.

> KEY TAKEAWAY
>
> *The same sentence can have many derivations. When a sentence has two different parse trees, the grammar is ambiguous — a serious problem.*

WORKED EXAMPLE

Parse Tree: Derivation Made Visual

Grammar: $E -> E + E$ | $E times E$ | $i d$

- **Root**: The start symbol E sits at the top of every parse tree.
- **Internal Nodes**: Each non-terminal expansion becomes an internal node with children equal to the RHS symbols.
- **Leaves**: The terminals, read left-to-right, form the derived string.
- **Ambiguity**: If a string has two distinct parse trees, the grammar is ambiguous — the compiler cannot decide meaning.

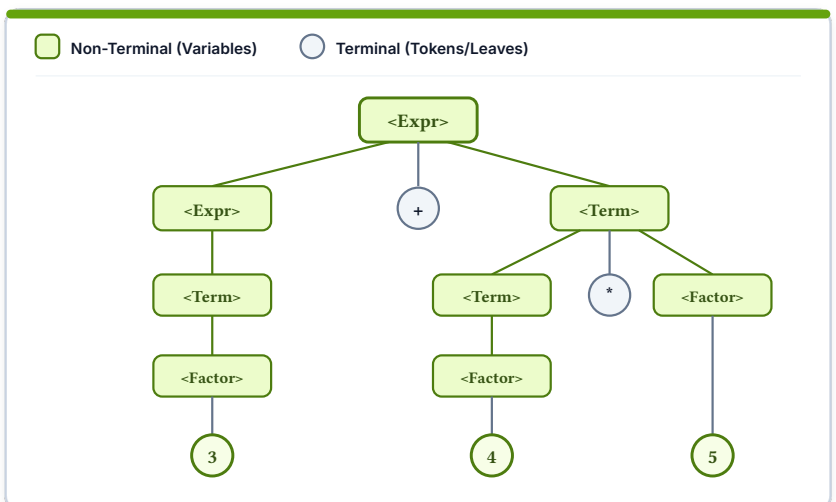


Figure: Two parse trees for `id + id * id` — the classic ambiguity example.

AMBIGUITY

Ambiguity: When Grammar Fails

Two trees, one string, no way to decide

- **Definition**: A grammar is ambiguous if some string has two or more distinct parse trees.
- **The Classic Case**: $E \to E + E \mid E \times E \mid id$ — `id + id * id` has two trees.
- **The Consequence**: The compiler does not know whether to add first or multiply first — different results!
- **The Fixes**: Introduce precedence layers, enforce associativity, or use a different grammar.

> KEY TAKEAWAY
>
> *Ambiguity is not a property of a string — it is a property of the grammar. The same language can have both ambiguous and unambiguous grammars.*

AMBIGUITY

Two Trees for One String

`id + id * id` — which tree is correct?

- **Left Tree**: Multiplication at the top — multiplication has lower precedence than addition.
- **Right Tree**: Addition at the top — multiplication has higher precedence (the conventional answer).
- **Precedence**: A rule that says which operator binds tighter — resolved by grammar layering.
- **Associativity**: A rule for equal precedence — left or right — resolved by recursive rule shape.

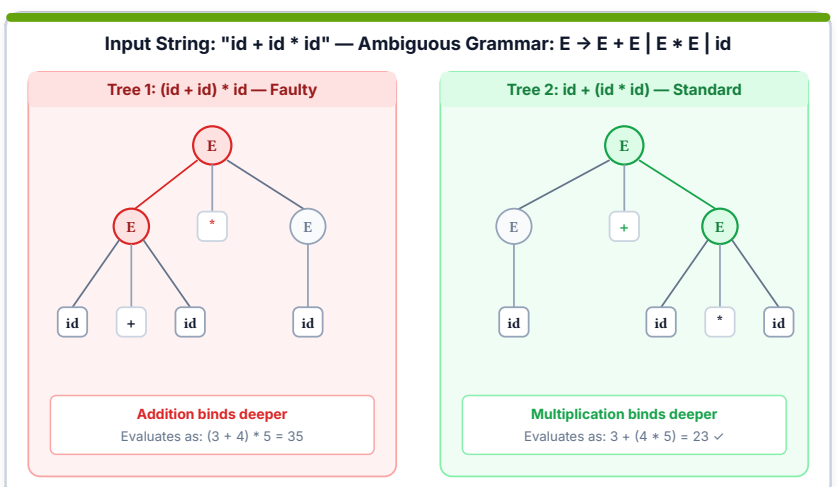


*Figure: Left tree groups (id + id) * id; right tree groups id + (id * id). Different semantics.*

GRAMMAR NOTATION

BNF: The Standard Notation

Backus-Naur Form — the practical face of context-free grammars

- **Notation**: `A ::= X Y | Z` — non-terminals in angle brackets, `::=` means ‘is defined as’, `|` means ‘or’.
- **Example**: `<expr> ::= <expr> '+' <term> | <term>` and `<term> ::= 'id'`.
- **Recursion**: Rules may refer to themselves — this is what lets a grammar describe infinitely many strings.
- **Readability Cost**: BNF gets verbose fast. Repetition requires explicit recursive rules, which bloats the spec.

> KEY TAKEAWAY
>
> *BNF is exactly a Context-Free Grammar (Chomsky Type 2) written in a human-friendly syntax.*

GRAMMAR NOTATION

BNF in Practice: A Tiny Language

Arithmetic expressions in pure BNF

- **Rule 1**: `<expr> ::= <expr> '+' <term> | <term>` — an expression is a sum of terms.
- **Rule 2**: `<term> ::= <term> '*' <factor> | <factor>` — a term is a product of factors.
- **Rule 3**: `<factor> ::= '(' <expr> ')' | <number>` — a factor is a parenthesized expr or a number.
- **Rule 4**: `<number> ::= <digit> | <number> <digit>` — numbers are built from digits.

> KEY TAKEAWAY
>
> *Four rules describe infinitely many arithmetic expressions. This is the compression power of formal grammars.*

GRAMMAR NOTATION

BNF vs EBNF

Verbose clarity vs compact convenience

BNF (Plain)

- **Repetition**: Requires explicit recursive rules — e.g. $A \to Y\,A \mid \varepsilon$.
- **Optionality**: Requires two alternative rules.
- **Strength**: Strict CFG form; directly feeds into theory and proofs.

EBNF (Extended)

- **Repetition**: Curly braces around Y means zero-or-more — no recursion needed.
- **Optionality**: Square brackets around Y means optional in one symbol.
- **Strength**: Compact, readable — used in every modern language spec (Java, Python, SQL).

Synthesis: EBNF is not more powerful than BNF — any EBNF grammar converts mechanically to BNF. It is purely a notational convenience.

GRAMMAR NOTATION

EBNF Operators Cheat Sheet

Five symbols that compress any grammar

- **Curly Braces { X }**: Zero or more repetitions of X — replaces tail recursion.
- **Square Brackets [X]**: Optional X (zero or one) — replaces two-rule alternatives.
- **Parentheses (X Y)**: Grouping — treat X Y as one unit.
- **Vertical Bar X | Y**: Alternation — same as BNF's choice operator.

> KEY TAKEAWAY
>
> *Convert { X } to a new non-terminal $A \to XA \mid \varepsilon$. Convert [X] to $A \to X \mid \varepsilon$. EBNF collapses to BNF mechanically.*

IMPLEMENTATION

A Real Grammar, In Code Form

```
# Grammar: expr ::= term { ("+" | "-") term }
#          term ::= factor { ("*" | "/") factor }
#          factor ::= NUMBER | "(" expr ")"

def parse_expr(tokens):
    left = parse_term(tokens)
    while peek(tokens) in ('+', '-'):
        op = consume(tokens)
        right = parse_term(tokens)
        left = BinOp(op, left, right)
    return left

def parse_term(tokens):
    left = parse_factor(tokens)
    while peek(tokens) in ('*', '/'):
        op = consume(tokens)
        right = parse_factor(tokens)
        left = BinOp(op, left, right)
    return left
```

- **One Method Per Rule**: Each grammar rule becomes one parser function — direct translation from EBNF to code.
- **Layered Precedence**: expr to term to factor enforces that * binds tighter than +, resolving ambiguity structurally.
- **Recursion = Stack**: The parser's call stack is the Pushdown Automaton (PDA) that recognizes the language.

AUTOMATA

From Grammar to Automaton

Every grammar has a matching machine

- **Type 3 (Regular)**: Grammar pairs with a Finite Automaton — no memory beyond current state.
- **Type 2 (Context-Free)**: Grammar pairs with a Pushdown Automaton — adds a stack for nesting.
- **Type 1 (Context-Sensitive)**: Grammar pairs with a Linear-Bounded Automaton — tape bounded by input length.
- **Type 0 (Unrestricted)**: Grammar pairs with a Turing Machine — infinite tape.

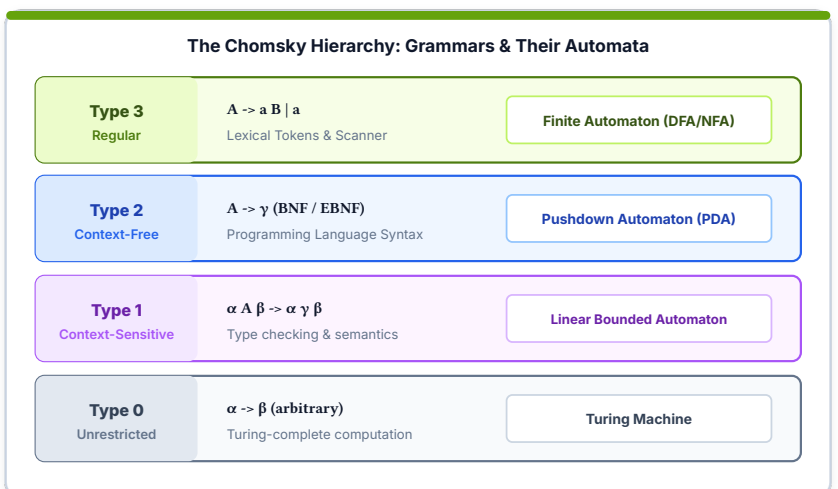


Figure: Chomsky hierarchy — grammar types paired with recognizing automata.

CHOMSKY HIERARCHY

Where Formal Grammars Sit in the Hierarchy

Type 0
unrestricted

Recursively Enumerable

Recognized by Turing Machines; undecidable in general.

Type 1
context-sensitive

Length-Bounded Rules

Recognized by Linear-Bounded Automata.

Type 2
context-free

CFG — BNF/EBNF Live Here

Recognized by Pushdown Automata; used for parsing.

Type 3 regular

Regex / Finite Automata

Used for tokenizing; cannot match balanced nesting.

Key Point: Programming language syntax lives in Type 2 (CFG). Token patterns live in Type 3 (regular). Both are formal grammars — just with different expressive power.

CLASSIFICATION

Context-Free vs Context-Sensitive

Why your compiler needs both

Context-Free (Type 2)

- **Rule Shape**: LHS is a single non-terminal — no surrounding context.
- **Strength**: Fast, deterministic parsers (LL, LR) exist.
- **Limitation**: Cannot enforce declaration-before-use or type rules.

Context-Sensitive (Type 1)

- **Rule Shape**: $\alpha A \beta \rightarrow \alpha \gamma \beta$ — context matters.
- **Strength**: Can model variable scoping, matching, and cross-references.
- **Limitation**: Parsing is expensive (PSPACE-complete); rarely used in practice.

Synthesis: Compilers split the work: CFG handles structure, and a separate semantic analyzer (symbol tables, type checkers) handles context.

EDGE CASES

What Grammars Cannot Express

The limits of context-free power

- **Declaration Before Use**: 'Every variable must be declared before use' is context-sensitive — a CFG cannot enforce it.
- **Type Checking**: 'Adding an int to a string is illegal' is a semantic rule, not a syntactic one.
- **Matching Counts Across Categories**: a^n b^n c^n is not context-free — a CFG can match two counts but not three.
- **Solution**: Compilers handle these in the semantic analyzer (symbol tables, type checkers), not the parser.

> KEY TAKEAWAY
>
> *Syntax is what a grammar can check. Semantics is what a grammar cannot. That boundary is exactly where the parser ends and the semantic analyzer begins.*

SCOPE OF GRAMMAR

What Grammar Covers vs Doesn't

Drawing the boundary clearly

- **Grammar covers**: Token sequences, nesting structure, operator precedence, statement forms, block structure.
- **Grammar does not cover**: Name resolution, type compatibility, aliasing, lifetime, side effects, evaluation order.
- **Why the Split**: Keeping the parser fast is worth pushing harder checks into a separate pass.
- **Practical Rule**: If it needs a symbol table, it is not grammar's job.

> KEY TAKEAWAY
>
> *Every real compiler is a pipeline: lexer (Type 3), parser (Type 2), semantic analyzer (rules grammar cannot express). Grammar defines only the first two stages.*

GRAMMAR DESIGN

Designing an Unambiguous Grammar

Three techniques that always work

- **Precedence Layering**: Introduce one non-terminal per precedence level — E for +, T for *, F for atoms.
- **Associativity Encoding**: Left-associative uses left recursion ($E \to E + T$); right-associative uses right recursion.
- **Eliminating Common Prefixes**: Factor $A \to \alpha\beta \mid \alpha\gamma$ into $A \to \alpha A'$, $A' \to \beta \mid \gamma$ — needed for LL(1).

> KEY TAKEAWAY
>
> *Unambiguous grammars are not harder to write — they are harder to write casually. Layer, don't lump.*

GRAMMAR TRANSFORMATIONS

Rewriting Grammars for Parsers

Same language, different shape

- **Left Recursion Elimination**: $A \to A\alpha \mid \beta$ becomes $A \to \beta A'$, $A' \to \alpha A' \mid \varepsilon$ — needed for LL parsers.
- **Left Factoring**: $A \to \alpha\beta \mid \alpha\gamma$ becomes $A \to \alpha A'$, $A' \to \beta \mid \gamma$ — needed for LL(1).
- **Preserving Language**: Both transforms keep the generated language identical — only the derivation structure changes.
- **Cost**: The new grammar may be less readable, and the parse tree may need post-processing.

> KEY TAKEAWAY
>
> *A grammar is a **specification**. A parser is a **mechanism**. Rewriting grammars bridges the two without changing the language.*

TRANSFORMATIONS

Left Recursion Elimination in Code

```
# Before (left recursive — cannot be parsed by LL):
#   E -> E + T | T
#   T -> T times F | F
#   F -> id

# After (right recursive — LL-friendly):
#   E  -> T E'
#   E' -> + T E' | epsilon
#   T  -> F T'
#   T' -> * F T' | epsilon
#   F  -> id

def parse_E(tokens):
    left = parse_T(tokens)
    return parse_E_prime(tokens, left)

def parse_E_prime(tokens, left):
    while peek(tokens) == '+':
        consume(tokens)
        right = parse_T(tokens)
        left = BinOp('+', left, right)
    return left
```

- **Rule-for-Rule**: Each rewritten rule becomes one method — the shape mirrors the transformation.
- **Loop Instead of Recursion**: The tail recursion of the rewritten grammar becomes a while loop.
- **Same Language**: Both grammars accept identical strings; only the derivation structure changed.

PARSING

Top-Down vs Bottom-Up Parsing

Two strategies from the same grammar

- **Top-Down (LL)**: Start at S, predict which rule to apply, descend to leaves — recursive descent or table-driven.
- **Bottom-Up (LR)**: Start at tokens, reduce to larger structures, ascend to S — shift-reduce parsers.
- **LL Strength**: Simple to write by hand; requires left-recursion elimination and left factoring.
- **LR Strength**: Handles more grammars (including left recursion); harder to write by hand but tools (Bison) generate them.

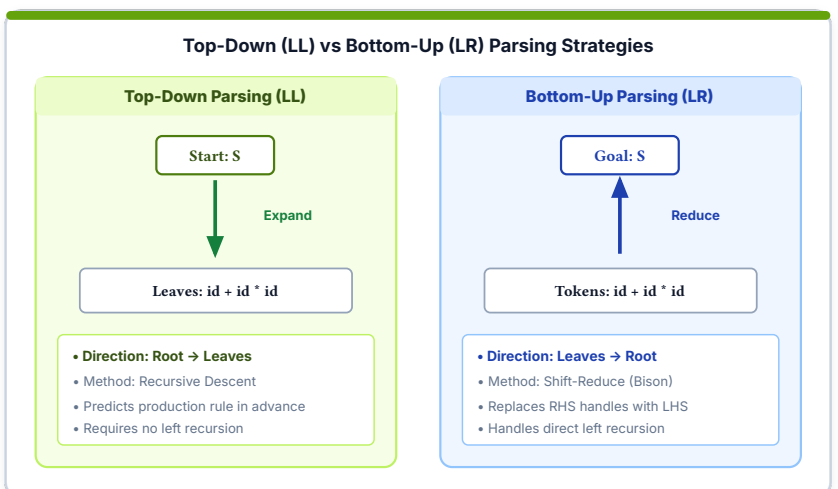


Figure: Top-down builds the tree from the root; bottom-up builds it from the leaves.

PARSING

LL(1) Parsing: The Table-Driven Approach

Predict the rule from one lookahead token

- **FIRST Set**: The set of terminals that can begin a string derived from a non-terminal — tells you which rule to pick.
- **FOLLOW Set**: The set of terminals that can appear immediately after a non-terminal — used when a rule can derive ε.
- **LL(1) Table**: Entry $M[A, a]$ gives the production to apply when the parser is at non-terminal A and sees lookahead a.
- **Determinism**: If any cell has more than one entry, the grammar is not LL(1) — needs rewriting.

> KEY TAKEAWAY
>
> *LL(1) parsers are the workhorse of hand-written recursive descent. Compute FIRST and FOLLOW once, then parsing is O(n).*

PARSING

Parser Family at a Glance

LL(1) top-down

Predictive, Table-Driven

One lookahead token; simple, fast, restricted grammars.

LL(k) top-down

Generalized Lookahead

k tokens lookahead; more grammars, larger tables.

LR(1) bottom-up

Canonical LR

Powerful but large tables; handles most real grammars.

LALR(1) bottom-up

Look-Ahead LR

Used by Yacc/Bison; smaller tables, nearly LR(1) power.

Practical Rule: Hand-written parsers use LL(1). Generated parsers use LALR(1). Both are grammar-driven and both are O(n).

TRADE-OFFS

Hand-Written vs Generated Parsers

Control vs convenience

Hand-Written (Recursive Descent)

- **Strength**: Full control over error messages and recovery.
- **Strength**: Easy to debug; direct grammar-to-code mapping.
- **Limitation**: Must eliminate left recursion; repetitive code.

Generated (ANTLR, Bison)

- **Strength**: Write grammar, get parser — fast iteration.
- **Strength**: LALR/LR handles left recursion directly.
- **Limitation**: Error messages are tool-generated, often cryptic.

Synthesis: Production compilers often mix both: generated parser for speed of development, custom recovery layers for quality of diagnostics.

REAL-WORLD

Grammars You Already Use

Formal grammars hide in every spec you read

- **JSON**: A tiny CFG — objects, arrays, strings, numbers — specified in EBNF in RFC 8259.
- **Python**: The reference grammar lives in `Grammar/Grammar` — pure BNF, used by the CPython parser.
- **SQL**: The SQL standard defines the grammar in BNF-like notation — hundreds of productions.
- **HTTP**: The request/response grammar is defined in ABNF (a variant of BNF) in RFC 7230.

> KEY TAKEAWAY
>
> *Whenever you read a language spec and see ::= or { }, you are reading a formal grammar — the same concepts from this lecture, in production.*

HISTORY

A Brief History of Grammar Notation

From Panini to ANTLR

- **500 BCE — Panini**: Sanskrit grammar with 4000 rules — the earliest formal grammar.
- **1956 — Chomsky**: The hierarchy that classified grammars by expressive power.
- **1959–1960 — Backus & Naur**: BNF introduced for ALGOL 58/60 — the first programming language grammar.
- **1970s–present — Yacc, Bison, ANTLR**: Parser generators turn grammars into executable code.

> KEY TAKEAWAY
>
> *Formal grammar is one of the oldest ideas in computer science — and one of the most reusable. The same theory runs every compiler today.*

SUMMARY

Key Takeaways

What to remember from this lecture

- **Grammar = Recipe**: A finite rule set that generates an infinite language via substitution (derivation).
- **Four Components**: $G = (V, T, P, S)$ — non-terminals, terminals, productions, start symbol.
- **Two Directions**: Generation (grammar to string) and recognition (string to grammar) are the same relationship viewed both ways.
- **Ambiguity**: Two parse trees for one string — fixed by layering precedence and associativity.

> KEY TAKEAWAY
>
> *Grammar defines syntax. The parser recognizes it. The semantic analyzer handles everything grammar cannot. This division is the entire compiler frontend.*

SYNTHESIS

Mental Model: Grammar as a Lens

One idea, four views

- **As a Definition**: A grammar is a finite recipe that defines an infinite language by substitution.
- **As a Generator**: Apply rules forward to produce valid strings — this is derivation.
- **As a Recognizer**: Invert the rules to check if a string is valid — this is parsing.
- **As a Compiler Artifact**: Grammar drives the lexer (Type 3), parser (Type 2), and AST shape.

> KEY TAKEAWAY
>
> *Once you see every language — natural or programming — as a set of strings plus a grammar that generates them, the entire compiler pipeline becomes an exercise in bookkeeping.*

LOOKING AHEAD

Where We Go Next

From grammar to working parsers

- **Lexical Analysis**: Regex, NFAs, DFAs — the machine that turns characters into tokens.
- **Top-Down Parsing**: Recursive descent, FIRST/FOLLOW sets, LL(1) tables.
- **Bottom-Up Parsing**: Shift-reduce, LR(0), SLR, LALR, CLR.
- **Semantic Analysis**: Symbol tables, attribute grammars, type checking — the rules grammar cannot express.

> KEY TAKEAWAY
>
> *Formal grammar is the vocabulary. Parsing is the grammar in motion. Semantic analysis is the grammar's blind spot. Master all three and you understand the compiler frontend.*

EXERCISES

Practice Problems

Test your understanding

- **Derivation**: Derive 'aaabbb' from $S \to aSb \mid \varepsilon$ using leftmost derivation. How many steps?
- **Ambiguity**: Show that $E \to E + E \mid E \times E \mid id$ is ambiguous by drawing two parse trees for `id + id * id`.
- **BNF to EBNF**: Rewrite `<list> ::= <item> | <item> ',' <list>` in EBNF.
- **Design**: Write a BNF grammar for 'a list of zero or more integers separated by commas, enclosed in square brackets'.

> KEY TAKEAWAY
>
> *Do at least the first two. Grammar intuition comes from doing derivations by hand, not from reading them.*

FURTHER READING

Where to Go Deeper

References for the curious

- **Dragon Book**: Aho, Lam, Sethi, Ullman — **Compilers: Principles, Techniques, and Tools**, Chapters 2–4.
- **Chomsky (1956)**: 'Three Models for the Description of Language' — the original hierarchy paper.
- **ANTLR Reference**: Terence Parr — **The Definitive ANTLR 4 Reference** — grammars in practice.
- **Crafting Interpreters**: Robert Nystrom — free online; builds a complete language from grammar to execution.

> KEY TAKEAWAY
>
> *The Dragon Book is the canonical reference. Crafting Interpreters is the best hands-on companion. Read both in parallel.*